

Lab12 – Resources Requirements

Resource requests and limits

You are tasked with creating a Pod for running an application in container. During application development, you ran a load test for figuring out the minimum amount of resources needed and the maximum amount of resources the application is allowed to grow to. Define those resource requests and limits for the Pod.

1. Define a Pod named `hello-world` running the container image `bmuschko/nodejs-hello-world:1.0.0`. The container exposes the port 3000.

```
vagrant@kubernetes-control-plane:~/lab07$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab07$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
spec:
  containers:
  - image: bmuschko/nodejs-hello-world:1.0.0
    name: hello
    ports:
    - name: nodejs-port
      containerPort: 3000

vagrant@kubernetes-control-plane:~/lab07$
vagrant@kubernetes-control-plane:~/lab07$
```

2. Add a Volume of type `emptyDir` and mount it the container path `/var/log`.

```
vagrant@kubernetes-control-plane:~/lab07$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab07$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
spec:
  containers:
  - image: bmuschko/nodejs-hello-world:1.0.0
    name: hello
    ports:
    - name: nodejs-port
      containerPort: 3000
    volumeMounts:
    - name: log-volume
      mountPath: "/var/log"
  volumes:
  - name: log-volume
    emptyDir: {}

vagrant@kubernetes-control-plane:~/lab07$
vagrant@kubernetes-control-plane:~/lab07$
```

3. For the container, specify the following minimum number of resources as follows:

- CPU: 100m
- Memory: 500Mi
- Ephemeral storage: 1Gi

And the following maximum number of resources as follows:

- Memory: 500Mi
- Ephemeral storage: 2Gi

```
vagrant@kubernetes-control-plane:~/lab07$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab07$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello
spec:
  containers:
  - image: bmuschko/nodejs-hello-world:1.0.0
    name: hello
    ports:
    - name: nodejs-port
      containerPort: 3000
    volumeMounts:
    - name: log-volume
      mountPath: "/var/log"
    resources:
      requests:
        cpu: 100m
        memory: 500Mi
        ephemeral-storage: 1Gi
      limits:
        memory: 500Mi
        ephemeral-storage: 2Gi
  volumes:
  - name: log-volume
    emptyDir: {}

vagrant@kubernetes-control-plane:~/lab07$
```

4. Create the Pod from the YAML manifest. Inspect the Pod details. Which node does the Pod run on?

```
vagrant@kubernetes-control-plane:~/lab07$ kubectl apply -f pod.yaml
pod/hello created
vagrant@kubernetes-control-plane:~/lab07$ kubectl get nodes
vagrant@kubernetes-control-plane:~/lab07$ kubectl get pod hello -o wide
```

NAME	READY	STATUS	ROLES	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
hello	0/1	ContainerCreating		0	30s	<none>	kubernetes-node1	<none>	<none>

```
vagrant@kubernetes-control-plane:~/lab07$ kubectl describe pod hello
Name:
Namespace:
Priority:
Service Account:
Node:
Start Time:
Labels:
```

```

Limits:
  ephemeral-storage: 2Gi
  memory: 500Mi
Requests:
  cpu: 100m
  ephemeral-storage: 1Gi
  memory: 500Mi
Environment:
  <none>
Mounts:

```

Resource quota

In this exercise, you will create a ResourceQuota with specific CPU and memory limits for a new namespace. Pods created in the namespace will have to adhere to those limits.

5. Create a new Pod that exceeds the limits of the resource quota requirements e.g. by defining 1Gi of memory but stays below the CPU e.g. 0.5. Write down the error message.

```

vagrant@kubernetes-control-plane:~/lab$ kubectl create namespace rq-demo
namespace/rq-demo created
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f resourcequota.yaml --namespace=rq-demo
resourcequota/app created
vagrant@kubernetes-control-plane:~/lab$ kubectl describe quota --namespace=rq-demo
Name:         app
Namespace:    rq-demo
Resource      Used  Hard
-----
pods          0    2
requests.cpu  0    2
requests.memory 0    500Mi
vagrant@kubernetes-control-plane:~/lab$ cat pod-quota.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - image: nginx
    name: mypod
    resources:
      requests:
        cpu: "0.5"
        memory: "1Gi"
    restartPolicy: Never
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod.yaml --namespace=rq-demo
error: the path "pod.yaml" does not exist
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod-quota.yaml --namespace=rq-demo
Error from server (Forbidden): error when creating "pod-quota.yaml": pods "mypod" is forbidden: exceeded quota: app, requested: requests.memory=1Gi, used: requests.memory=0, limited: requests.memory=500Mi
vagrant@kubernetes-control-plane:~/lab$

```

6. Change the request limits to fulfill the requirements to ensure that the Pod could be created successfully. Write down the output of the command that renders the used amount of resources for the namespace.

```

vagrant@k8s-control-plane:~/lab$ vim pod-quota.yaml
vagrant@k8s-control-plane:~/lab$ cat pod-quota.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - image: nginx
    name: mypod
    resources:
      requests:
        cpu: "0.5"
        memory: "255Mi"
      restartPolicy: Never

vagrant@k8s-control-plane:~/lab$ kubectl apply -f pod-quota.yaml --namespace=rq-demo
pod/mypod created
vagrant@k8s-control-plane:~/lab$ kubectl describe quota --namespace=rq-demo
Name:          app
Namespace:     rq-demo
Resource       Used    Hard
-----
pods           1      2
requests.cpu   500m   2
requests.memory 255Mi  500Mi
vagrant@k8s-control-plane:~/lab$

```

Limit Range

A LimitRange can restrict resource consumption for Pods in a namespace, and assign default computing resource if no resource requirements have been defined. You will practice the effects of a LimitRange on the creation of a Pod in different scenarios.

7. Inspect the YAML manifest definition in the file `setup.yaml` (./setup.yaml). Create the objects from the YAML manifest file.

```

vagrant@k8s-control-plane:~/lab$ cat setup.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: d92
---
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-limit-range
  namespace: d92
spec:
  limits:
  - min:
      cpu: 200m
    max:
      cpu: 500m
    type: Container

vagrant@k8s-control-plane:~/lab$ kubectl apply -f setup.yaml
namespace/d92 created
limitrange/cpu-limit-range created
vagrant@k8s-control-plane:~/lab$ kubectl describe limitrange cpu-limit-range -n d92
Name:          cpu-limit-range
Namespace:     d92
Type           Resource  Min    Max    Default Request  Default Limit  Max Limit/Request Ratio
-----
Container      cpu       200m   500m   500m             500m           -
vagrant@k8s-control-plane:~/lab$

```

8. Create a new Pod named `pod-without-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` without any resource requirements. Inspect the Pod details. What resource definitions do you expect to be assigned?

```
vagrant@kubernetes-control-plane:~/lab$ cat pod-without-resource-requirements.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-without-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx

vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod-without-resource-requirements.yaml
pod/pod-without-resource-requirements created
vagrant@kubernetes-control-plane:~/lab$
```

A Pod without specifying resource requirements will use the default request and limit defined by LimitRange, in this case 500m.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl describe pod pod-without-resource-requirements -n d92
Name:          pod-without-resource-requirements
Namespace:     d92
Priority:       0
Service Account: default
Node:          kube-node1/192.168.56.11
Start Time:    Sun, 13 Aug 2023 12:51:40 +0000
Labels:        <none>
Annotations:   kubernetes.io/limit-ranger: LimitRanger plugin set: cpu request for container nginx; cpu limit for container nginx
Status:        Running
IP:            10.44.0.1
IPs:           IP: 10.44.0.1
Containers:
  nginx:
    Container ID:   containerd://ad003618a16b159072d60adcab706beb3f5f09bf59bde85d16c7ab8da97aba0
    Image:          nginx:1.23.4-alpine
    Image ID:       docker.io/library/nginx@sha256:02ffd439b71d9ea9408e449b568f65c0bbb94bebd8750f1d80231ab6496008e
    Port:          <none>
    Host Port:     <none>
    State:          Running
      Started:      Sun, 13 Aug 2023 12:52:10 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          500m
    Requests:
      cpu:          500m
    Environment:    <none>
```

9. Create a new Pod named `pod-with-more-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 400m and limits of 1.5. What runtime behavior do you expect to see?

```
vagrant@kubernetes-control-plane:~/lab$ cat pod-with-more-cpu-resource-requirements.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-more-cpu-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
    resources:
      requests:
        cpu: 400m
      limits:
        cpu: 1.5

vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod-with-more-cpu-resource-requirements.yaml
Error from server (Forbidden): error when creating "pod-with-more-cpu-resource-requirements.yaml": pods "pod-with-more-cpu-resource-requirements" is forbidden: maximum cpu usage per Container is 500m, but limit is 1500m
vagrant@kubernetes-control-plane:~/lab$
```

10. Create a new Pod named `pod-with-less-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 350m and limits of 400m. What runtime behavior do you expect to see?

```
vagrant@kubernetes-control-plane:~/lab$ cat pod-with-less-cpu-resource-requirements.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-less-cpu-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
    resources:
      requests:
        cpu: 350m
      limits:
        cpu: 400m

vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f pod-with-less-cpu-resource-requirements.yaml
pod/pod-with-less-cpu-resource-requirements created
vagrant@kubernetes-control-plane:~/lab$
```

The CPU resource request and limit fits within the boundaries of the LimitRange.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl describe pod pod-with-less-cpu-resource-requirements -n d92
Name: pod-with-less-cpu-resource-requirements
Namespace: d92
Priority: 0
Service Account: default
Node: kube-node2/192.168.56.12
Start Time: Sun, 13 Aug 2023 12:56:35 +0000
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.36.0.2
IPs:
  IP: 10.36.0.2
Containers:
  nginx:
    Container ID: containerd://1b26ce0b16f741d72ceb4142b3467b03a485d2e7da2a242623277057ab13c9de
    Image: nginx:1.23.4-alpine
    Image ID: docker.io/library/nginx@sha256:02ff439b71d9ea9408e449b568f65c0bbb94bebd8750f1d80231ab6496008e
    Port: <none>
    Host Port: <none>
    State: Running
      Started: Sun, 13 Aug 2023 12:57:25 +0000
    Ready: True
    Restart Count: 0
    Limits:
      cpu: 400m
    Requests:
      cpu: 350m
    Environment: <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-85trf (ro)
Conditions:
  Type Status
  Initialized True
```